# 09-17: lexical analysis (part 1)

As discussed previously, a compiler's first job is to decide if the source program is well-formed, whether it is included in the source language and thus has a well-defined meaning. If the program text does not make sense in the source language, the compiler must emit an error. The so-called analysis phase is responsible for this task, and it is divided to a number of subtasks, each checking more and more complex properties of the program. The first such analysis step is **lexical analysis** (aka lexical analyser, lexer, scanner), which checks if the letters (characters) in the source text can be recognized as a series of "words" (lexemes). Simply put, lexical analysis is an algorithm whose input is a list of characters (text) and whose output is the list of lexical symbols (tokens) recognized in the text. Lexical analysis processes the characters of the input text from left to right, one by one. In case there is an unsupported character found in the text, or there is an unclosed multi-line comment, we get a lexical error and the compilation terminates; on the other hand, if there are no errors, we get the sequence of lexical symbols (tokens).

## Lexical categories

What kind of symbols do we yield? Well, in an average language, think of keywords, identifiers, or literals: these are all lexical categories, sorts of words that may appear in a computer program text. Obviously, the lexical categories are specific to each language.

There are also white spaces, but most languages' lexical analysers simply ignore them as they do not influence the meaning of the program, and they are there only for the human reader. However, there are also layout-sensitive languages, where indentation does contribute to syntax and semantics, in such languages white spaces are turned into lexical symbols.

## A concrete example

For example, in a C-like language, you would identify lexical categories for keywords, identifiers, literals, parenthesis, as well as operator symbols, separator and terminator symbols. Scanning the following simple text, we would expect it to be chopped as follows.

```
if ( var1 == 12 ) var2 = 34 ;
-- - ---- -- -- - ---- - -- -

if   : keyword
(    : opening parenthesis
var1 : identifier
==   : equality check
12   : number literal
)    : closing parenthesis
var2 : identifier
=    : assignment symbol
34   : number literal
;    : end-of-statement symbol
```

The lexical analysis not only finds the lexemes, but also attaches a kind to them (e.g., keyword, identifier, etc.). Note that at this phase, analysis is not concerned with grammatical rules, variable scopes or types; the lexical analysis is only checking if the text can be understood as a sequence of lexical symbols; whether those symbols make sense grammatically and semantically are the responsibility of further steps.

# Principles of lexical analysis

We highlight here that the scanner algorithm is to read the input text from left to right, and assess if a prefix belongs to any lexical categories. There are special scenarios where such a prefix belongs to multiple lexical categories, and sometimes there are multiple such prefixes that can be accepted. To mitigate these, we apply two basic **principles**:

- Maximal munch (longest match): the scanner, reading from left to right, will recognize the longest possible token starting at the current position in the text. Note that this does not mean finding a global maximum by reading the whole file, rather it is the maximal prefix of the text which can be recognized as a lexeme.
- Priority: when a lexeme (a symbol recognized by the lexical analyser) belongs to multiple lexical categories, it is recognized as the one with the higher priority. Therefore, lexical categories are always ordered (in flex, you define this order with the order of appearance of the regular expressions in the file).

**Example.** Suppose that in a language, *while* is a keyword, but it could be an identifier as well (provided that any non-empty sequence of letters could be an identifier). When the scanner of this language reads the prefix "while", it has to decide whether it recognizes it as a keyword or an identifier; typically, we prefer to recognize such lexemes as keywords, which we can achieve by assigning a higher (relative) priority to keywords. Another issue solved by the second principle is when a keyword appears as a prefix to a longer identifier, for example, the repetition of the keyword. For instance, the lexeme "whilewhile" (without any separators or whitespace) is to be decoded as a single identifier, not two keywords, and the longest match principle will guarantee that.

# Definition of lexical categories

How do we decide if a prefix of the rest of the input text belongs to a lexical category? Well, lexical categories are themselves formal languages, in fact, sub-languages to the programming language. Fortunately, they are fairly simple, almost always regular; hence, lexical categories can be defined with regular expressions.

Regular expressions define regular languages which just five different pattern constructors: empty string, singleton (a single alphabet symbol), union, concatenation, closure (in increasing order of precedence). There are other operations defined on top of these, such as positive closure, optional, and interval (range). For details, please see the Cornell handout.

As an example, let us turn the following informal description of the format of identifiers into a declarative, formal definition: "identifiers consist of letters, digits and underscores, but they need to start with a letter".

This would be defined with interval, concatenation and closure:

```
[a-zA-Z][a-zA-Z0-9_]*
```

# Implementation of lexical analysis

Lexical analysers are rarely written ad-hoc because it would be tedious and error-prone. Rather, once lexical categories are **specified** with regular expressions, they are converted to deterministic automata, serving as the basis for the lexical analysis **implementation**.

The conversion happens in three main steps (for details, please see the handouts):

- With a recursive algorithm, we convert the regular expression into a non-deterministic automaton (see Thompson's construction algorithm)
- We eliminate epsilon-transitions by merging epsilon-steps with non-epsilon steps
- With subset construction, we merge states to avoid ambiguous transitions

For an example, consider the following (very simple) languages over the alphabet {a,b}:

- a|b
- a*b

After converting these regular expressions to NFA and eliminating epsilon-transitions, we already get two DFAs that can recognize the corresponding languages (two a letters and a series of a's followed by a b, respectively); see **this drawing (https://canvas.elte.hu/courses/66503/files/4518841/download?wrap=1)** for the two NFAs and DFAs recognizing the languages of the regexes.

We can easily encode these DFAs in C++ as simple loops processing the input symbols and maintaining a state. For example, the implementation of the above two automata could look like this (returning -1 on reject and returning the maximal matching length on accept). See *dfa_dfa_tokenA* and *dfa_dfa_tokenB* in **this file (https://canvas.elte.hu/courses/66503/files/4518842/download?wrap=1)** .

However, being able to recognize the individual lexical categories is not a lexer yet; we need to incorporate the principles of priority and maximal munch, and couple the automata with a main logic driving the acceptance and carrying on with the rest of the text --- or reporting errors when there's no match. You can inspect the *scan* method in **this file (https://canvas.elte.hu/courses/66503/files/4518842/download?wrap=1)** .

Note that this is still a very naive implementation of lexical analysis, but it does encode the DFAs and uses them to check for every prefix whether a token can be recognized, and handles acceptance as well as errors.

In the following lecture, we will smarten this solution by merging the automatons, and by employing an optimisation that allows the lexer to skip rerunning the automata on long strings with hopeless states.

- Cornell handout: **https://www.cs.cornell.edu/courses/cs4120/2023sp/notes/#lexing (https://www.cs.cornell.edu/courses/cs4120/2023sp/notes/#lexing)**

- Stanford handout: **https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/040%20Lexical%20Analysis.pdf (https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/040%20Lexical%20Analysis.pdf)**